

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_5

5. Prototype Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Prototype pattern.

GoF Definition

It specifies the kinds of objects to create using a prototypical instance and creates new objects by copying this prototype.

Concept

This pattern provides an alternative method for instantiating new objects by copying or cloning an instance of an existing object. Thus, you can avoid the expense of creating a new instance using this concept. Why is this important? There are various reasons. Here are some examples:

- You may need some resources to make an instance, but these resources may not be easily available to you.
- The construction of a new instance from scratch is a time-consuming operation.

If you examine the intent of the pattern (the GoF definition) closely, you'll see that the core idea of this pattern is to create an object that is based on another object. This existing object acts as a template for the new object.

When you examine the code for this pattern, in general, you'll see that there is an abstract class or interface that plays the role of an abstract prototype. This abstract prototype contains a cloning method that is implemented by the concrete prototype(s). A client can create a new object by asking a prototype to clone itself. You'll see that the upcoming program (Demonstration 1) follows the same approach.

Real-Life Example

Suppose you have a master copy of a valuable document. You need to incorporate some changes to it in order to analyze the effect of the changes. In this case, you can make a photocopy of the original document and make the changes on the photocopied document.

Computer World Example

Let's assume that you already have a stable application. In the future, you may want to modify the application with some small changes. You must start with a copy of your original application, make the changes, and then analyze further. Surely you do not want to start coding from scratch to merely make a change; this would cost you time and money.

In Java programming, the `clone()` method of the `Object` class is an example of the Prototype pattern. This method can create and return a copy of an existing object. In this context, your class needs to be `Cloneable`. Let's see what Javadoc says about the `clone()` method. Here are some important points for your immediate reference:

- It creates and returns a copy of this object. The precise meaning of “copy” may depend on the class of the object.
- By convention, the returned object should be obtained by calling `super.clone`.
- By convention, the object returned by this method should be independent of this object (which is being cloned). To achieve this independence, it may be necessary to modify one or more fields of the object returned by `super.clone` before returning it.
- The method `clone` for class `Object` performs a specific cloning operation. First, if the class of this object does not implement the interface `Cloneable`, then a `CloneNotSupportedException` is thrown.

Implementation

This example follows the structure shown in Figure [5-1](#).

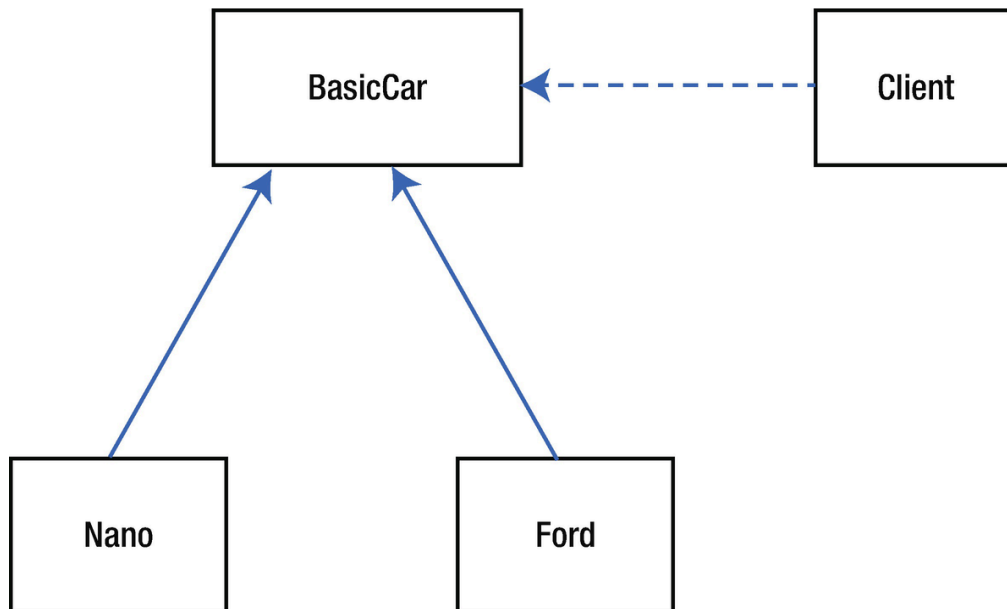


Figure 5-1 Prototype example

Here `BasicCar` is the prototype. It is an abstract class that has an abstract method called `clone()`. `Nano` and `Ford` are the concrete classes (i.e., concrete prototypes) that inherit from `BasicCar`. Both concrete classes implement the `clone()` method. Initially, you create a `BasicCar` object with a default price. Later you modify that price as per the model. The `Client.java` file represents the client in this implementation.

Inside the `Nano` class constructor and the `Ford` class constructor, you add a random price to the basic price to generate the on-road price of a car. Here is the `Nano` class:

```

import java.util.Random;
class Nano extends BasicCar {
    public Nano(String modelName) {
        this.modelName = modelName;
        // The base price for a Nano car
        basePrice = 5000;
        // Set the on-Road price
        onRoadPrice = basePrice + (new Random()).nextInt(1000);
    }
    @Override
    public BasicCar clone() throws CloneNotSupportedException {
        return (Nano) super.clone();
    }
    @Override
    public String toString()
    {
        return "Model:"+modelName+"\nPrice: "+onRoadPrice;
    }
}

```

The other concrete prototype, `Ford`, has a similar structure. To understand the prototype pattern, only one concrete prototype is sufficient. So, if you want, you can drop/ignore any one of these concrete prototypes to reduce the code size.

Inside the client code, the actual objects (which were instantiated directly using the `new` operator) use the `toString()` method to show the model and price of the car. There's also an optional method `printCarDetail()` inside the client code; it's used to increase the on-road price of a cloned car. This may give you an idea of how to edit a cloned object. Here is the method description:

```
private static void printCarDetail(BasicCar car) {  
    System.out.println("Editing a cloned model:");  
    System.out.println("Model: " + car.modelName);  
    // Editing the on-Road price of a car  
    // This is an optional step  
    car.onRoadPrice += 100;  
    System.out.println("It's on-road price:$" + car.onRoadPrice);  
}
```

Lastly and most importantly, you'll see the use of the `clone()` method in the upcoming examples. It is defined in the `Object` class. You know that the `Object` class is the root of the class hierarchy. Every class has `Object` as a superclass in Java.

Class Diagram

Figure [5-2](#) shows the class diagram. In the upcoming demonstration, you'll see that `BasicCar` is an abstract class that implements the `java.lang.Cloneable` interface.

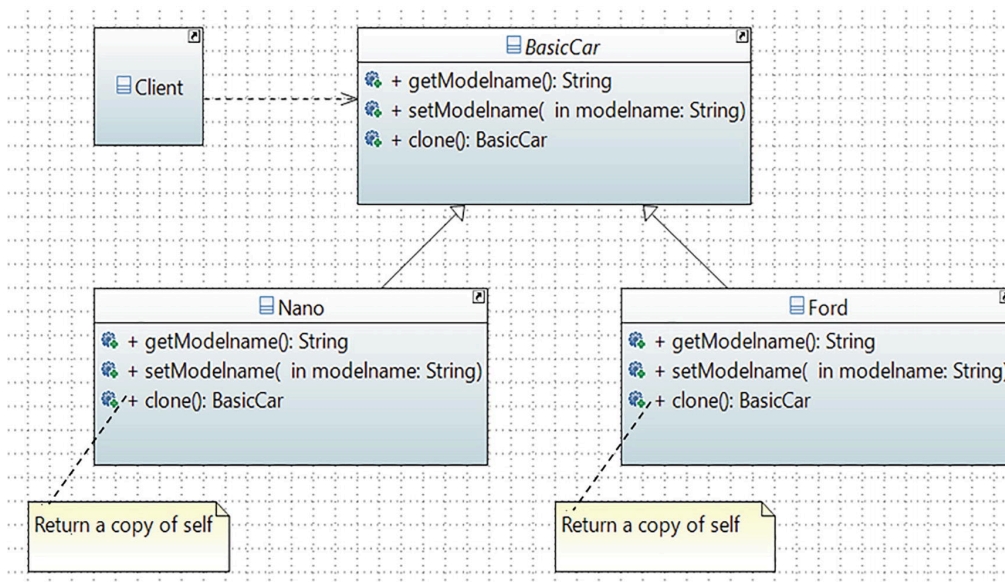


Figure 5-2 Class diagram

Package Explorer View

Figure [5-3](#) shows the high-level structure of the program.

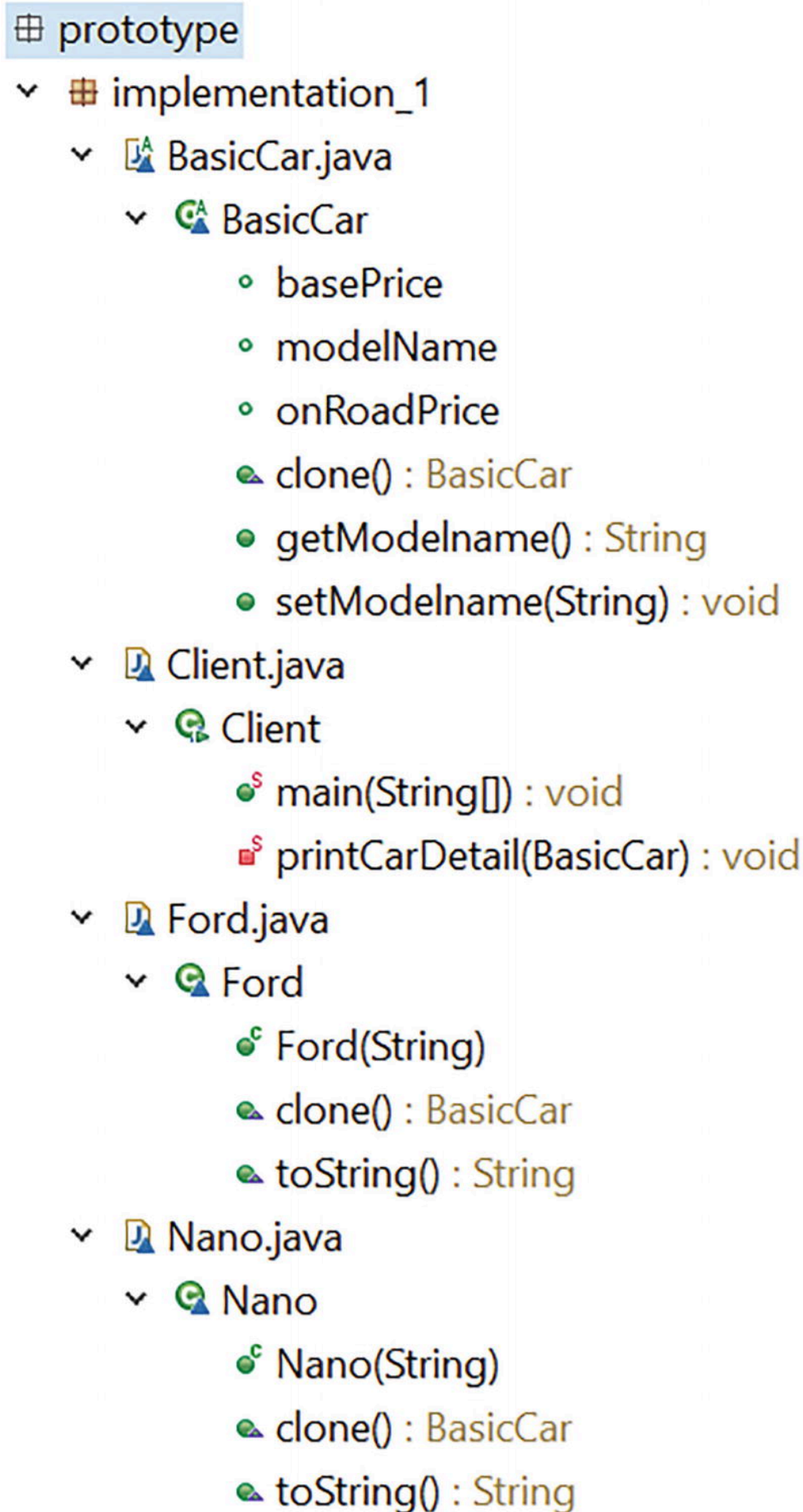


Figure 5-3 Package Explorer view

Demonstration 1

Here's the implementation. All classes are stored inside the package

`jdp3e.prototype.implementation_1`.

```
// BasicCar.java
abstract class BasicCar implements Cloneable {
    public String modelName;
    public int basePrice=0,onRoadPrice=0;
    public String getModelname() {
        return modelName;
    }
    public void setModelname(String modelname) {
        this.modelName = modelname;
    }

    public BasicCar clone() throws CloneNotSupportedException {
        return (BasicCar)super.clone();
    }
}

// Nano.java
import java.util.Random;
class Nano extends BasicCar {
    public Nano(String modelName) {
        this.modelName = modelName;
        // The base price for a Nano car
        basePrice = 5000;
        // Set the on-road price
        onRoadPrice = basePrice + (new
            Random()).nextInt(1000);
    }
    @Override
    public BasicCar clone() throws CloneNotSupportedException {
        return (Nano) super.clone();
    }
    @Override
    public String toString()
    {
        return "Model: "+modelName+"\nPrice: "+onRoadPrice;
    }
}

// Ford.java
import java.util.Random;
class Ford extends BasicCar {
    public Ford(String modelName) {
        this.modelName = modelName;
        // The base price for a Ford car
```

```

        basePrice = 40000;
        // Set the on-road price
        onRoadPrice=basePrice+ (new Random()).nextInt(1000);
    }
    @Override

    public BasicCar clone() throws CloneNotSupportedException {
        return (Ford) super.clone();
    }
    @Override
    public String toString() {
        return "Model: "+modelName+"\nPrice: "+onRoadPrice;
    }
}
// Client.java
class Client {
    public static void main(String[] args) throws
        CloneNotSupportedException {
        System.out.println("***Prototype Pattern Demo***\n");
        // Working with a Nano car
        BasicCar nano = new Nano("Nano XM624 cc");
        System.out.println(nano);
        System.out.println("-----");
        // Getting a cloned version of Nano
        BasicCar clonedCar;
        clonedCar = nano.clone();
        // Working with the cloned Nano
        printCarDetail(clonedCar);
        System.out.println("-----\n");
        // Working with a Ford car copy
        BasicCar ford = new Ford("Ford Aspire");
        System.out.println(ford);
        System.out.println("-----");
        // Getting a cloned version of Ford
        clonedCar = ford.clone();
        // Working with the cloned Ford

        printCarDetail(clonedCar);
        System.out.println("-----\n");
    }
    private static void printCarDetail(BasicCar car) {
        System.out.println("Editing a cloned model:");
        System.out.println("Model: " + car.modelName);
        // Editing the on-Road price of a car

```



```
// This is an optional step
car.onRoadPrice += 100;
System.out.println("It's on-road price: $" + car.onRoadPrice);
}
}
```

Output

Here is a possible output:

```
***Prototype Pattern Demo***
Model: Nano XM624 cc
Price: 5003
-----
Editing a cloned model:
Model: Nano XM624 cc
Its on-road price: $5103
-----
Model: Ford Aspire
Price: 40020

-----
Editing a cloned model:
Model: Ford Aspire
Its on-road price: $40120
-----
```

Note You may see a different price in your system because the code generates a random price in the constructors of the `Nano` and `Ford` classes. This example also ensures that the price of a `Ford` is greater than a `Nano`.

Modified Implementation

Notice that in demonstration 1, before making a clone, the client instantiated the objects as follows:

```
BasicCar nano = new Nano("Nano XM624 cc");
BasicCar ford = new Ford("Ford Aspire");
```

This is fine, but in some examples of the Prototype pattern, you may notice an additional participant that creates the prototypes and supplies

them to a client. Experts often like this approach because it hides the complexity of creating new instances inside the client. So let's introduce a `CarMaker` class and modify the implementation.

Class Diagram

Figure 5-4 shows some important changes in this modified class diagram. This time there is a new class called `CarMaker` that is used to get a new car or a cloned car. Notice that this class contains two `BasicCar` references inside it, `nano` and `ford`.

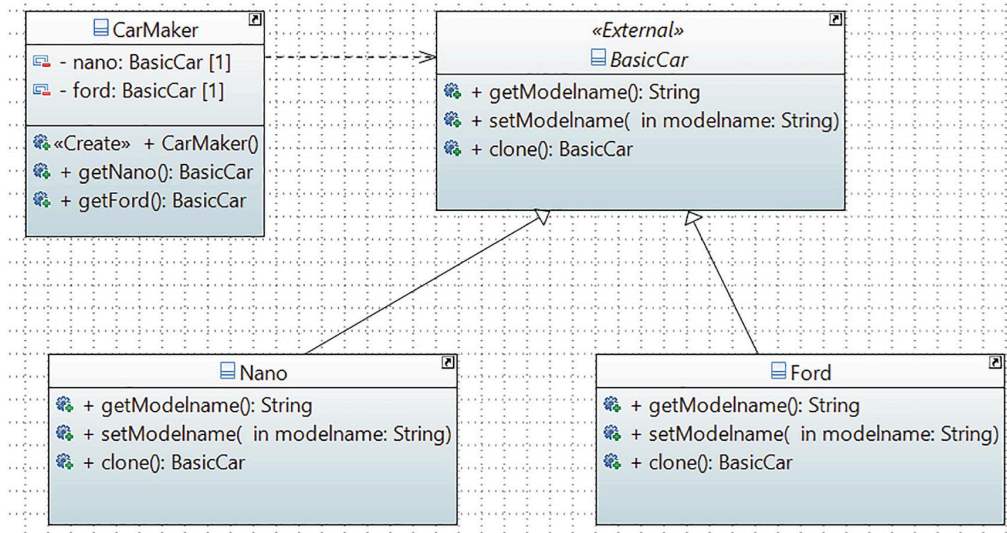


Figure 5-4 Key changes in the class diagram for demonstration 2

Demonstration 2

To demonstrate the modified implementation, add the following class called `CarMaker`:

```

// CarMaker.java
class CarMaker {
    BasicCar nano, ford;
    public CarMaker() {
        nano = new Nano("Nano XM624 cc");
        ford = new Ford("Ford Aspire");
    }
    public BasicCar getNano() throws
        CloneNotSupportedException {
        return nano.clone();
    }
    public BasicCar getFord() throws
        CloneNotSupportedException {
        return ford.clone();
    }
}
  
```

```
}
```

When you have this class, your client code can be modified as follows. The important changes are in bold.

```
// Client.java
class Client {
    public static void main(String[] args) throws
        CloneNotSupportedException {
        System.out.println("***Prototype Pattern Modified Demo***\n");
        CarMaker carMaker = new CarMaker();
        // Working with a Nano car
        BasicCar nano = carMaker.getNano();
        System.out.println(nano);
        System.out.println("-----");
        // Getting a cloned version of Nano
        BasicCar clonedCar;
        clonedCar = nano.clone();
        // Working with the cloned Nano
        printCarDetail(clonedCar);
        System.out.println("-----\n");
        // Working with a Ford car copy
        BasicCar ford = carMaker.getFord();
        System.out.println(ford);
        System.out.println("-----");
        // Getting a cloned version of Ford
        clonedCar = ford.clone();
        // Working with the cloned Ford
        printCarDetail(clonedCar);
        System.out.println("-----\n");
    }
    private static void printCarDetail(BasicCar car) {
        System.out.println("Editing a cloned model:");
        System.out.println("Model: " + car.modelName);
        // Editing the on-Road price of a car
        // This is an optional step

        car.onRoadPrice += 100;
        System.out.println("It's on-road price: $" + car.onRoadPrice);
    }
}
```

To avoid repetition, I did not show the other participants, which were already shown in demonstration 1. When you download the source code

from the Apress website, refer to the `implementation_2` folder inside the `prototype` folder to get the complete program.

Output

Here is a possible output:

```
***Prototype Pattern Modified Demo***
Model: Nano XM624 cc
Price: 5356
-----
Editing a cloned model:
Model: Nano XM624 cc
Its on-road price: $5456
-----
Model: Ford Aspire
Price: 40501
-----
Editing a cloned model:
Model: Ford Aspire
Its on-road price: $40601
```

Analysis

This output is similar to the previous output and there is no magic. The `CarMaker` class serves your needs, but there is a potential drawback to it. Notice that you initialize the cars inside the constructor of `CarMaker`. As a result, it always creates instances of both car types when this class is initialized. If you want to implement a lazy initialization, you can modify the `CarMaker` class as follows:

```
/**
 * @author Vaskaran Sarcar
 * Alternative version of the CarMaker class.
 */
class CarMaker {
    BasicCar nano, ford;
    public CarMaker() {
    }
    public BasicCar getNano() throws
        CloneNotSupportedException {
        if (nano!=null) {
            // A Nano instance was created earlier.
            // Returning a clone of it.
            return nano.clone();
        } else {
```

```

        /*
         * Create a nano for the first time and return it.
         */
        nano = new Nano("Nano XM624 cc");
        return nano;
    }
}

public BasicCar getFord() throws
    CloneNotSupportedException {

    if (ford!=null){
        // A Ford instance was created earlier.
        // Returning a clone of it.
        return ford.clone();
    } else {
        /*
         * Create a Ford for the first
         * time and return it.
         */
        nano = new Ford("Ford Aspire");
        return nano;
    }
}
}

```

Further Improvements

These are not the ultimate modifications. Why? Let's go through the following points:

- In Chapter [7](#), when I discuss the Singleton pattern, you'll learn that in a multithreading environment you can produce additional objects when you check the `if` condition(s). Since you'll get the idea about the possible solutions also in Chapter [7](#), I won't repeat the same discussion here.
- In Chapter [8](#), when I discuss the Proxy pattern, I show you the usefulness of the package-private visibility. There you'll see that a client cannot directly make an instance of `ConcreteSubject` so it is forced to use the `ProxySubject` class. You can apply the same concept to prevent direct instantiation of a `Nano` or a `Ford` inside client code and force your client to use the `CarMaker` class only.

I believe that you now have a clear idea about the intent of this pattern.

Q&A Session

5.1 What are the advantages of using the Prototype design pattern?

Here are some important usages:

- You do not want to modify the existing object and experiment on it.
- You can include or discard products at runtime.
- In some contexts, you can create new instances at a cheaper cost.
- You can focus on the key activities rather than focusing on complicated instance creation processes. For example, once you ignore the complex object creation processes, you can simply start with cloning or copying objects and implementing the remaining parts.
- You want to examine a proposed behavior and its impact before you fully implement it in your application.

5.2 What are the challenges associated with using the Prototype design pattern?

Here are some challenges:

- Each subclass needs to implement the cloning or coping mechanism.
- Implementing the cloning mechanism can be challenging if the objects under consideration do not support copying or if there are circular references. For example, in Java, a class with `clone()` method needs to implement the `Cloneable` marker interface; otherwise, it will throw a `CloneNotSupportedException`.
- In this example, you use `super.clone()`. This `clone()` method performs a shallow copy, not a deep copy. (If you want to learn more, put your cursor on the eclipse editor and go through the detailed description). If you need a deep copy for your application, that can be expensive.

Note The class `Object` does not itself implement the interface `Cloneable`. So, if you call the `clone` method on an `Object` type, you will receive an exception at runtime.

5.3 Can you elaborate on the difference between a shallow copy and a deep copy?

The following discussion explains the difference between these two techniques.

NoteThe following discussion is added for the readers who are not aware of this difference. I assume that expert programmers are familiar with it. If you know the difference, you can skip the following discussion and continue reading the next Q&A.

Shallow Copy vs. Deep Copy

A shallow copy creates a new object and then copies the non-static fields from the original object to the new object. If there is a value type field in the original object, a bit-by-bit copy is performed. But if the field is a reference type, this method will copy the reference, not the actual object. Let's try to understand the mechanism with a simple diagram; see Figure 5-5. Suppose you have object X1 and it has a reference to another object, Y1. Again, object Y1 has a reference to object Z1.

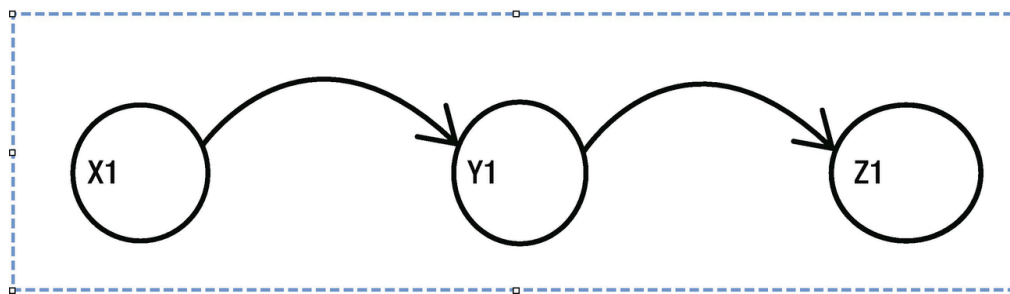


Figure 5-5 Before the shallow copy of the references

Now, with a shallow copy of X1, a new object (say, X2) will be created that will also have a reference to Y1 (see Figure 5-6).

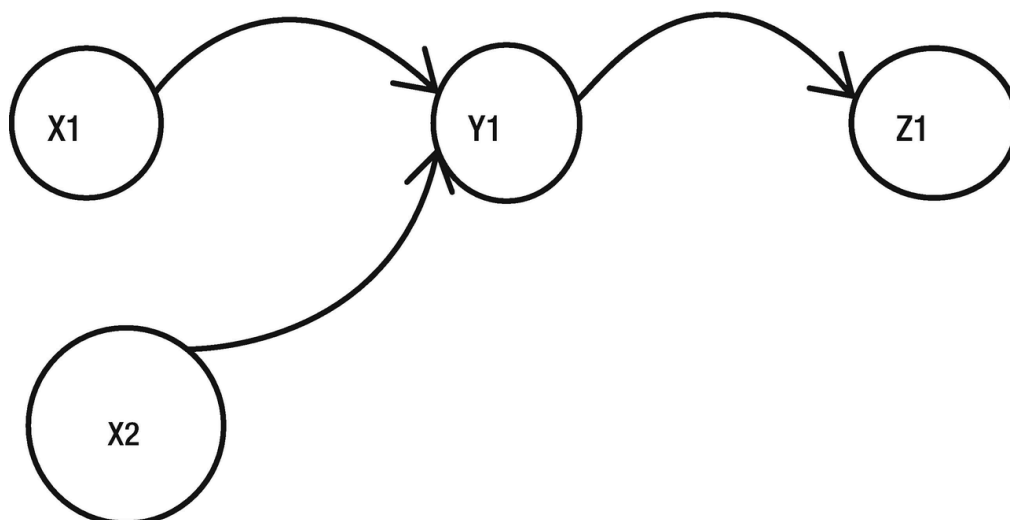


Figure 5-6 After a shallow copy of the reference

You use `clone()` in the previous implementation. It performs a shallow copy.

But for a deep copy of `X1`, a new object (say, `X3`) will be created, and `X3` will have a reference to the new object `Y3` that is a copy of `Y1`. Also, `Y3`, in turn, will have a reference to another new object, `Z3`, that is a copy of `Z1` (see Figure 5-7).

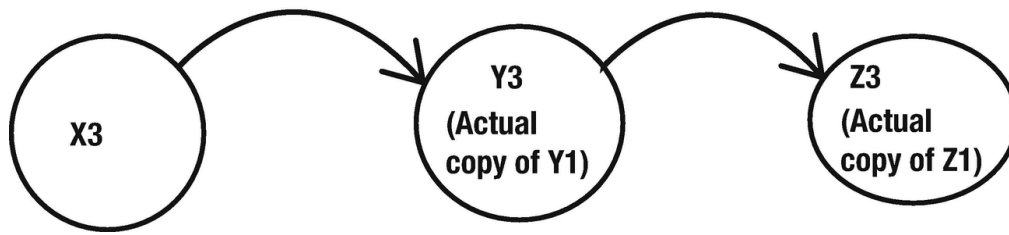


Figure 5-7 After a deep copy of the reference

Now consider the following demonstration for a better understanding.

Demonstration 3

This simple demonstration shows you the difference between a shallow copy and a deep copy. It also shows you the importance of a deep copy in certain situations. Here are the key characteristics of the program:

- This example has two classes, `Employee` and `EmpAddress`.
- `EmpAddress` has a single field called `Address`. It is used to set the address of an employee. The `Employee` class has three fields: `id`, `name`, and `empAddress` of types `int`, `String`, and `EmpAddress`, respectively.
- To create an `Employee` object, you need to pass an `id` of the employee and their name. At the same time, you need to pass the address.

```
EmpAddress initialAddress = new EmpAddress("21, abc Road,USA");
Employee emp = new Employee(1, "John", initialAddress);
```

- In the client code, first you create an `Employee` object `emp` and then you create another object `empClone` through cloning:

```
System.out.println("\nMaking a clone of emp now.");
Employee empClone = (Employee) emp.cloneEmployee();
```

- Later you change the values inside `empClone`.
- When you use a shallow copy, you'll see that as a side effect of this change, the address of the `emp` object also changes, but this is unwanted. (Remember that the basic idea of the Prototype pattern is straightforward: you should not change the original object when you work on a cloned copy of the object).

- In this upcoming example, initially the code for the deep copy is commented out so that you can see the effect of shallow copy only.

Now go through the demonstration. You'll find this program inside the package `jdp3e.prototype.shallow_vs_deep_copy`.

```
// EmpAddress.java
class EmpAddress implements Cloneable {
    String address;
    public EmpAddress(String address) {
        this.address = address;
    }
    @Override
    public String toString() {
        return this.address;
    }

    public EmpAddress cloneAddress() throws
        CloneNotSupportedException {
        // Shallow Copy
        return (EmpAddress) this.clone();
    }
}

// Employee.java
class Employee implements Cloneable {
    int id;
    String name;
    EmpAddress empAddress;
    public Employee(int id, String name, EmpAddress empAddress) {
        this.id = id;
        this.name = name;
        this.empAddress = empAddress;
    }
    @Override
    public String toString() {
        String empDetail = null;
        empDetail = "Employee detail: Id: " + id + " Name: "
            + name + " Address: " + empAddress;
        return empDetail;
    }
    public Employee cloneEmployee() throws
        CloneNotSupportedException {
        // For the shallow Copy
        return (Employee) this.clone();
        // For the deep copy
        // Employee employee = (Employee) this.clone();
    }
}
```

```

        // employee.empAddress = (EmpAddress) this.empAddress.
        // cloneAddress();
        // return employee;
    }

}

// Client.java
class Client {
    public static void main(String[] args) throws
        CloneNotSupportedException {
        System.out.println("***Shallow vs Deep Copy Demo.***\n");
        EmpAddress initialAddress = new EmpAddress("21, abc Road, USA");
        Employee emp = new Employee(1, "John", initialAddress);
        System.out.println("The original object is emp which is as follows:");
        System.out.println(emp);
        System.out.println("\nMaking a clone of emp now.");
        Employee empClone = (Employee) emp.cloneEmployee();
        System.out.println("The empClone object is as follows:");
        System.out.println(empClone);
        System.out.println("\n--Changing the detail of the cloned object now.-
        empClone.id = 10;
        empClone.name = "Sam";
        empClone.empAddress.address = "123, xyz Road, Canada";
        System.out.println("\nNow the emp object is as follows:");
        System.out.println(emp);
        System.out.println("\nAnd the emp1Clone object is as follows:");
        System.out.println(empClone);
    }

}

```

Output From Shallow Copy Implementation

Here is the output of the program:

```

***Shallow vs Deep Copy Demo.***
The original object is emp which is as follows:
Employee detail: Id: 1 Name: John Address: 21, abc Road, USA
Making a clone of emp now.
The empClone object is as follows:
Employee detail: Id: 1 Name: John Address: 21, abc Road, USA
--Changing the detail of the cloned object now.--

```

Now the emp object is as follows:

Employee detail: Id: 1 Name: John Address: **123, xyz Road, Canada**

And the emp1Clone object is as follows:

Employee detail: Id: 10 Name: Sam Address: **123, xyz Road, Canada**

Analysis

You can see an unwanted side effect. In the previous output, you can see that the address of the original object (emp) is modified due to the modification to the cloned object (emp1Clone). This happened because the original object and the cloned object both point to the same address and are not 100% disjointed. Figure 5-8 depicts the scenario better.

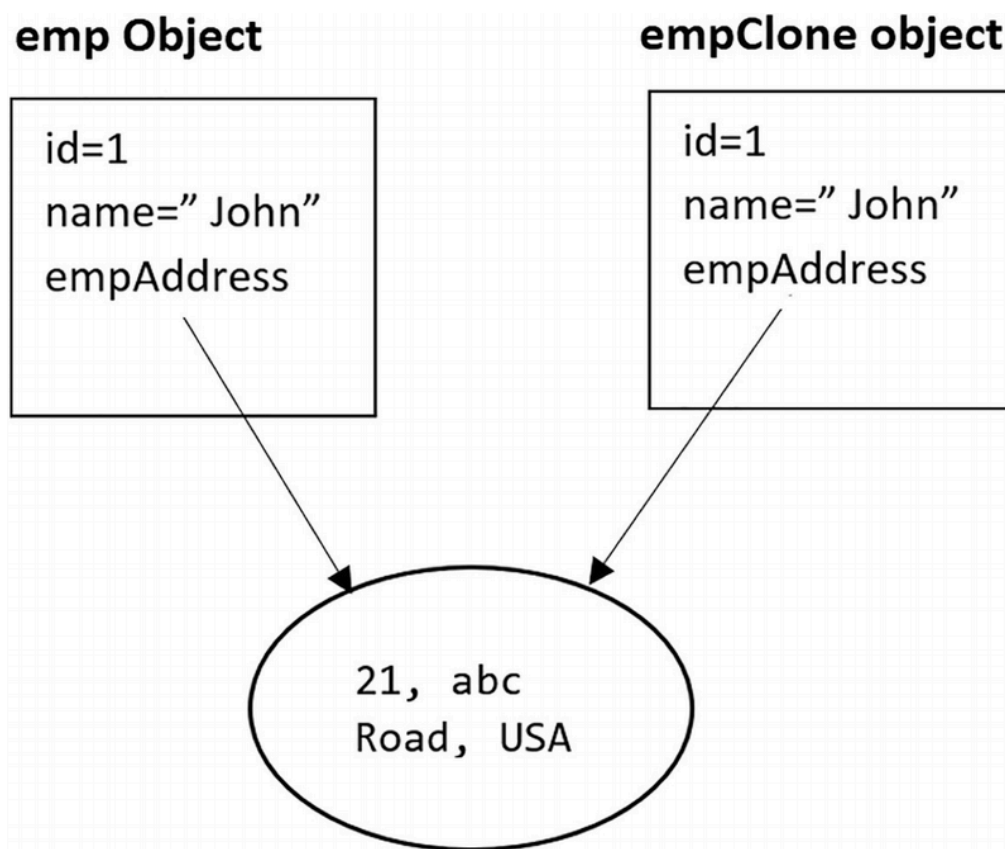


Figure 5-8 Shallow copy

Now let's experiment with a deep copy implementation. Let's modify the cloneEmployee() of the Employee class as follows. Uncomment the code for the deep copy and comment out the code for the shallow copy here. Notice the changes in bold.

```

public Employee cloneEmployee() throws CloneNotSupportedException {
    // For the shallow Copy
    // return (Employee) this.clone();
    // For the deep copy
    Employee employee = (Employee) this.clone();
    employee.empAddress = (EmpAddress) this.empAddress.cloneAddress();
}
  
```

```
        return employee;  
    }
```

Output From Deep Copy Implementation

Here is the modified output:

```
***Shallow vs Deep Copy Demo.***  
The original object  
  
    is emp which is as follows:  
Employee detail: Id: 1 Name: John Address: 21, abc Road, USA  
Making a clone of emp now.  
The empClone object is as follows:  
Employee detail: Id: 1 Name: John Address: 21, abc Road, USA  
--Changing the detail of the cloned object now.--  
Now the emp object is as follows:  
Employee detail: Id: 1 Name: John Address: 21, abc Road, USA  
And the emp1Clone object is as follows:  
Employee detail: Id: 10 Name: Sam Address: 123, xyz Road, Canada
```

Analysis

Notice that this time you don't see the unwanted side effect due to the modification to the `empClone` object. It is because the original object and the cloned object are totally different and independent of each other. Figure [5-9](#) depicts the scenario.

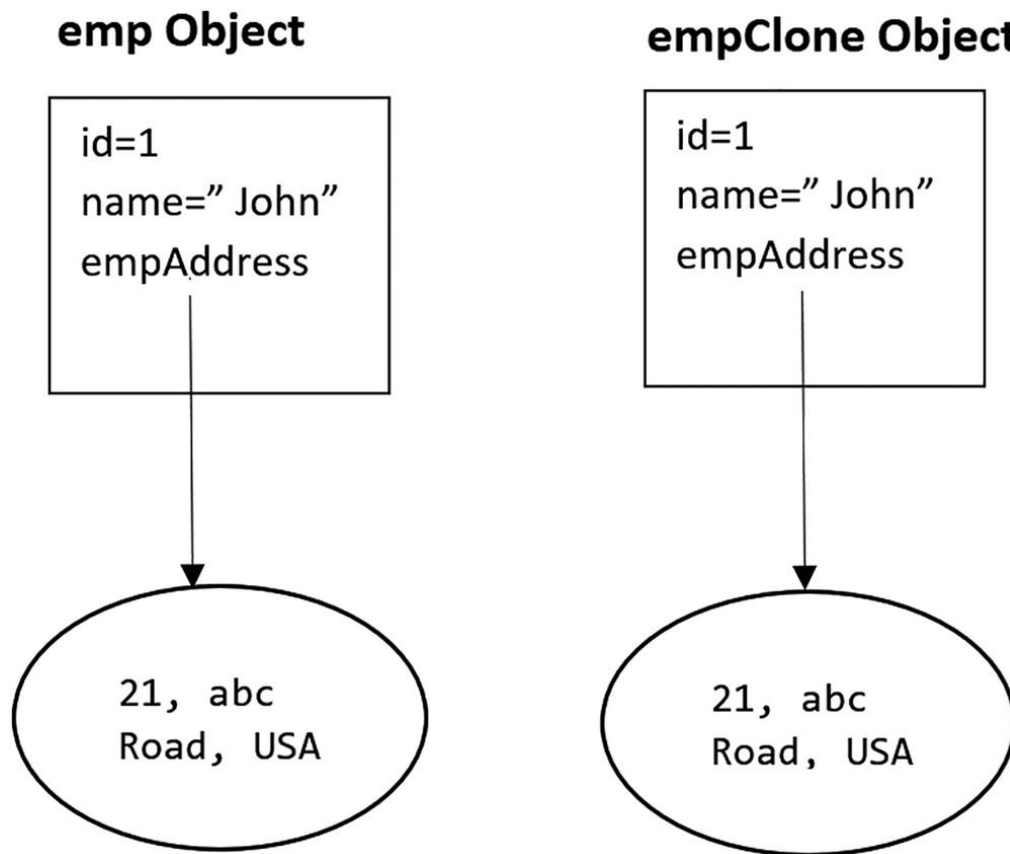


Figure 5-9 Deep copy

Q&A Session Continued

5.4 When should I choose a shallow copy over a deep copy (and vice versa)?

Here are the key differences:

- A shallow copy is faster and less expensive. It is always better if your target object has the primitive fields only.
- A deep copy is expensive and slow. But it is useful if your target object contains many fields that have references to other objects.

5.5 If I need to copy an object, I need to use the `clone()` method . Is the understanding correct?

No, there are alternatives available. For example, you can opt for a serialization mechanism when you implement a deep copy, or you can write your own copy constructor, etc. Each approach has its pros and cons. So, in the end, it is the developer's choice as to which approach fulfills their need. Many objects are really simple, and they do not contain references to other objects. So, to copy those objects, a simple shallow copy mechanism is sufficient.

5.6 In the Javadoc, the return type of the `clone()` method is `protected`. But in demonstration 1, it's made `public`. Is there any specific reason for this?

Yes. When you do this, you enhance its visibility to the maximum, which allows you to access the method from outside the package too. But your observation is correct. Since all parts of demonstration 1 are inside the same package, it was not absolutely necessary.

5.7 Can you show me an example that demonstrates the use of a copy constructor?

Since Java does not support a default copy constructor, you may need to write your own copy constructor. You can consider the following program (Demonstration 4) for your immediate reference.

Demonstration 4

In this example, the `Employee` and `EmpAddress` classes both have almost the same description as in the previous demonstration (Demonstration 3). The only difference is that this time, instead of the `clone()` method in the `Employee` class, you'll notice the presence of a copy constructor inside it. Let's proceed.

This time you are also using the same instance constructor:

```
public Employee(int id, String name, EmpAddress empAddress) {  
    this.id = id;  
    this.name = name;  
    this.empAddress = empAddress;  
}
```

It allows you to create an object of `Employee` as follows:

```
EmpAddress initialAddress = new EmpAddress("21, abc Road, USA");  
Employee emp = new Employee(1, "John", initialAddress);
```

As said before, in this `Employee` class, there is also a user-defined copy constructor, which is as follows:

```
// Copy Constructor  
public Employee(Employee originalEmployee) throws
```

```

CloneNotSupportedException {
    this.id = originalEmployee.id;
    this.name = originalEmployee.name;
    this.empAddress = originalEmployee.empAddress.cloneAddress();
}

```

You can see that using the copy constructor, you're copying both the simple type (Id, Name) and the reference type (EmpAddress). So, once an Employee object, say emp, is created, you can create another object named empClone from it using the following code:

```
Employee empClone= new Employee(emp);
```

As in the previous demonstration, once you create a copy (empClone) from the existing object (emp), you can make changes to the copied object for verification purposes, which is easy to understand. Here is the complete code, which is inside the package `jdp3e.prototype.implementation_4`:

```

// EmpAddress.java
class EmpAddress implements Cloneable {
    String address;
    public EmpAddress(String address) {
        this.address = address;
    }
    @Override
    public String toString() {
        return this.address;
    }
    public EmpAddress cloneAddress() throws
        CloneNotSupportedException {
        // Shallow Copy
        return (EmpAddress) this.clone();
    }
}

// Employee.java
class Employee implements Cloneable {
    int id;
    String name;
    EmpAddress empAddress;
    public Employee(int id, String name, EmpAddress
        empAddress) {
        this.id = id;
        this.name = name;
        this.empAddress = empAddress;
    }
    @Override

```

```

        public String toString() {

            String empDetail = null;
            empDetail = "Employee detail: Id: " + id + " Name: "
                + name + " Address: " + empAddress;
            return empDetail;
        }
        // Copy Constructor
        public Employee(Employee originalEmployee) throws
            CloneNotSupportedException {
            this.id = originalEmployee.id;
            this.name = originalEmployee.name;
            this.empAddress =
                originalEmployee.empAddress.cloneAddress();
        }
    }
    // CopyConstructorDemo.java
    class CopyConstructorDemo {
        public static void main(String[] args) throws CloneNotSupportedException {
            System.out.println("***A simple copy constructor demo***\n");
            EmpAddress initialAddress = new EmpAddress("21, abc Road, USA");
            Employee emp = new Employee(1, "John", initialAddress);
            System.out.println("The original object is emp which is as follows:");
            System.out.println(emp);
            System.out.println("\n Copying from emp to empCopy now.");
            Employee empCopy = new Employee(emp);
            System.out.println("The empCopy object is as follows:");
            System.out.println(empCopy);
            System.out.println("\n--Changing the detail of the copied object now.-");
            empCopy.id = 10;
            empCopy.name = "Sam";
            empCopy.empAddress.address = "123, xyz Road, Canada";
            System.out.println("\nNow the emp object is as follows:");
            System.out.println(emp);
            System.out.println("\nAnd the empCopy object is as follows:");
            System.out.println(empCopy);
        }
    }
}

```

Output

Here is the output. Notice the bold lines.


```
***A simple copy constructor demo***  
The original object is emp which is as follows:  
Employee detail: Id: 1 Name: John Address: 21, abc Road, USA  
Copying from emp to empCopy now.  
The empCopy object is as follows:  
Employee detail: Id: 1 Name: John Address: 21, abc Road, USA  
--Changing the detail of the copied object now.--  
Now the emp object is as follows:  
Employee detail: Id: 1 Name: John Address: 21, abc Road, USA  
And the empCopy object is as follows:  
Employee detail: Id: 10 Name: Sam Address: 123, xyz Road, Canada
```

Analysis

Notice the final portion of the output. It shows that when you incorporate a change, it is properly reflected in the copied object only.

This chapter showed you multiple implementations of prototype design patterns and discussed the difference between a shallow copy and a deep copy. You also learned the use of a user-defined copy constructor. Now you can move to the next chapter and continue learning another interesting pattern.

Previous chapter

[< 4. Abstract Factory Pattern](#)

Next chapter

[6. Builder Pattern >](#)